
CSE251A Programming Project 2: Coordinate Descent

Marsol Torrent, Sergi (UC San Diego)¹

Abstract

This project investigates coordinate descent methods for minimizing the logistic regression loss in a binary classification setting. We implement adaptive coordinate descent strategies based on the Gauss–Southwell rule and a normalized variant, and compare their convergence behavior against a random-feature coordinate descent baseline. Experimental results on the Wine dataset show that adaptive selection significantly accelerates convergence in terms of iterations while maintaining similar per-iteration computational complexity. We further extend the method to obtain sparse solutions by enforcing a cardinality constraint on the parameter vector. The proposed k -sparse coordinate descent algorithm demonstrates a trade-off between sparsity and optimality, with performance improving monotonically as the number of allowed non-zero coefficients increases.

1. Introduction

This project studies the unconstrained optimization problem

$$\min_{w \in \mathbb{R}^d} L(w), \quad (1)$$

where $L(w)$ is a differentiable loss function. In particular, we focus on minimizing the logistic regression loss using coordinate descent. While gradient-based methods such as gradient descent and stochastic gradient descent update all coordinates simultaneously, coordinate descent instead updates a single coordinate at a time. This often leads to simpler updates and can provide computational advantages, especially when the dimension is moderate or when coordinate-wise updates admit closed-form solutions.

The goal of this project is to design, implement, and evaluate an adaptive coordinate descent method for logistic regression. We compare its performance against a random-feature coordinate descent baseline and analyze convergence behavior. Additionally, we extend the method to obtain sparse solutions with a constraint on the number of non-zero coefficients.

1.1. Problem Setup

We consider binary logistic regression. Given data $\{(x_n, y_n)\}_{n=1}^N$ with feature vectors $x_n \in \mathbb{R}^d$ and labels

$y_n \in \{-1, +1\}$, the logistic loss is defined as (Bishop, 2006)

$$L(w) = \sum_{n=1}^N \log(1 + \exp(-y_n w^\top x_n)). \quad (2)$$

The objective is to find w that minimizes $L(w)$. In this project, we explore coordinate descent methods of the form:

- Select a coordinate $i \in \{1, \dots, d\}$.
- Update the corresponding parameter w_i while keeping all other coordinates fixed.

Two key design choices must be specified: (i) how to choose the coordinate at each iteration, and (ii) how to update the selected coordinate.

We implement and compare the following approaches:

- Adaptive coordinate descent using the Gauss–Southwell rule
- A normalized Gauss–Southwell variant
- Random-feature coordinate descent, where coordinates are selected uniformly at random.

Finally, we extend the method to enforce sparsity constraints of the form $\|w\|_0 \leq k$, where at most k coordinates are allowed to be non-zero. This enables investigation of sparse coordinate descent in the context of convex logistic loss.

1.2. Dataset

We evaluate the proposed coordinate descent methods on the Wine dataset from the UCI Machine Learning Repository (Aeberhard et al., 1994). The original dataset consists of 178 samples belonging to three distinct classes, each represented by 13 continuous features derived from chemical analysis of wines grown in the same region of Italy.

To obtain a binary classification problem suitable for logistic regression, we retain only samples from the first two classes, resulting in a dataset of 130 observations: 59 samples from class 1 and 71 samples from class 2. Each sample is represented by a 13-dimensional feature vector.

The class labels are converted to the binary set $\{-1, +1\}$, where samples from class 1 are assigned label $+1$ and samples from class 2 are assigned label -1 .

Prior to training, all features are standardized to have zero mean and unit variance. This preprocessing step improves numerical stability and ensures that coordinate-wise updates are not biased by differences in feature scale.

2. Coordinate Descent Method

Coordinate descent minimizes the objective function by updating one coordinate of w at a time while keeping the others fixed (Wright, 2015). At each iteration t , the method consists of two steps: (i) selecting a coordinate i_t , and (ii) updating the corresponding parameter w_{i_t} . Below we describe both components of our implementation.

2.1. Coordinate Selection Rule

The first design choice is how to select the coordinate to update. Let $\nabla_i L(w)$ denote the partial derivative of $L(w)$ with respect to coordinate w_i .

Gauss–Southwell (GS) rule. The classical Gauss–Southwell rule selects the coordinate with the largest magnitude partial derivative:

$$i_t = \arg \max_{i \in \{1, \dots, d\}} |\nabla_i L(w^{(t)})|. \quad (3)$$

Intuitively, this rule chooses the coordinate expected to yield the largest instantaneous decrease in the objective. This rule is known to improve convergence rates compared to uniformly random coordinate selection (Nutini et al., 2015).

Normalized Gauss–Southwell rule. Since different coordinates may have different curvature scales, we also consider a normalized variant:

$$i_t = \arg \max_{i \in \{1, \dots, d\}} \frac{|\nabla_i L(w^{(t)})|}{\sqrt{L_i(w^{(t)})}}, \quad (4)$$

where $L_i(w)$ is a coordinate-wise Lipschitz constant defined below. This normalization accounts for curvature differences across coordinates and prevents features with large scale from dominating the selection rule. Empirically, this variant converges faster than the unnormalized GS rule. For comparison, we also implement a random-feature coordinate descent baseline that selects i_t uniformly at random.

2.2. Coordinate Update Rule

Once a coordinate i_t is selected, we update it using a coordinate-wise quadratic upper bound on the logistic loss. Specifically, we perform the update

$$w_{i_t}^{(t+1)} = w_{i_t}^{(t)} - \frac{1}{L_{i_t}(w^{(t)})} \nabla_{i_t} L(w^{(t)}), \quad (5)$$

while keeping all other coordinates fixed.

For logistic regression, the second derivative of the loss with respect to coordinate w_i is given by

$$\frac{\partial^2 L(w)}{\partial w_i^2} = \sum_{n=1}^N x_{n,i}^2 \sigma(z_n)(1 - \sigma(z_n)), \quad (6)$$

where $z_n = y_n w^\top x_n$.

Since the sigmoid function satisfies $\sigma(u)(1 - \sigma(u)) \leq \frac{1}{4}$ for all u , this implies the uniform bound

$$\frac{\partial^2 L(w)}{\partial w_i^2} \leq \frac{1}{4} \sum_{n=1}^N x_{n,i}^2. \quad (7)$$

Therefore, a valid coordinate-wise Lipschitz constant is

$$L_i = \frac{1}{4} \sum_{n=1}^N x_{n,i}^2, \quad (8)$$

which is independent of w and guarantees that the quadratic upper bound used in the coordinate update is valid.

This choice of step size corresponds to minimizing a local quadratic upper bound on $L(w)$ along coordinate i , and avoids the need for line search. Such Lipschitz-based step sizes are commonly used in coordinate descent methods for smooth convex optimization (Nesterov, 2012). The update depends only on first-order derivatives and coordinate-wise curvature information.

2.3. Differentiability Requirements

Our method relies on computing coordinate-wise gradients $\nabla_i L(w)$ and Lipschitz constants $L_i(w)$. Therefore, the loss function must be differentiable with respect to each coordinate. In addition, Lipschitz continuity of the gradient is required to guarantee that the quadratic upper bound is valid.

Continuous second-order derivatives are not strictly required, since we do not explicitly compute the full Hessian. For logistic regression, the loss is smooth and twice differentiable, so these assumptions are satisfied.

2.4. Convergence Discussion

The logistic loss is convex in w . Moreover, its gradient is Lipschitz continuous on bounded domains. Under these conditions, coordinate descent with appropriate step sizes produces a monotonically decreasing sequence of objective values. Formal convergence guarantees for coordinate descent under these assumptions are well established in the literature (Wright, 2015).

Although the logistic loss is convex, when the data are linearly separable the infimum of the objective is not attained at any finite parameter vector. In this case, $L(w) \rightarrow 0$ as $\|w\| \rightarrow \infty$, and therefore the optimization problem does not admit a finite global minimizer. Consequently, coordinate descent generates a sequence of iterates whose loss values decrease toward $\inf_w L(w) = 0$, rather than converging to a finite minimizing parameter vector. In the present dataset, the classes are linearly separable, and the unregularized logistic loss approaches zero as $\|w\| \rightarrow \infty$. In practice, both adaptive coordinate descent variants converge numerically to a loss value indistinguishable from the baseline optimal loss L^* .

Empirically, the normalized Gauss–Southwell rule achieves faster convergence (in terms of iterations) than both the unnormalized GS rule and random-feature coordinate descent, while preserving monotonic decrease of the objective.

3. Experimental Methods

3.1. Baseline Logistic Regression

All optimization methods are evaluated on the full standardized training dataset, and all reported loss values correspond to training loss. As a reference solution, we first minimize the logistic loss using a standard batch solver. We train a logistic regression model using the `LogisticRegression` implementation from the `scikit-learn` library in an effectively unregularized setting.

Specifically, we set the inverse regularization parameter to $C = 10^{10}$ and disable the intercept term in order to match the unconstrained optimization problem. Optimization is performed using the `lbfgs` solver with `max_iter` = 10000 and tolerance `tol` = 10^{-12} .

This procedure yields a training accuracy of 100% on the standardized dataset. The resulting optimal loss value is

$$L^* = 9.54 \times 10^{-9}, \quad (9)$$

which is numerically close to zero. Since the data are linearly separable, the logistic loss does not admit a finite minimizer and approaches zero as $\|w\| \rightarrow \infty$. Therefore, this value should be interpreted as the stopping loss returned by the batch solver rather than the true optimal loss. This behavior is expected since the two classes in the filtered dataset are linearly separable, implying that the logistic loss approaches zero as $\|w\| \rightarrow \infty$.

The obtained loss value L^* is used as a baseline for evaluating the convergence of coordinate descent methods.

3.2. Adaptive Coordinate Descent

We implement two adaptive coordinate descent methods based on Gauss–Southwell selection rules: the standard GS rule and the normalized GS rule described in Section 2.

At each iteration, the selected coordinate is updated according to

$$w_i \leftarrow w_i - \frac{1}{L_i} \nabla_i L(w), \quad (10)$$

where L_i is the coordinate-wise Lipschitz constant. Both variants are run for 5000 iterations starting from the zero initialization.

For the GS rule, the coordinate with the largest magnitude partial derivative is selected. For the normalized GS rule, the selection criterion is scaled by $\sqrt{L_i}$ to account for curvature differences across features.

In our experiments, both adaptive methods converge rapidly to a loss value numerically equal to L^* . The normalized GS rule consistently reaches target loss thresholds in fewer iterations than the unnormalized GS rule. For example, the

normalized variant attains loss values below 1.0, 0.1, and 0.01 after 19, 44, and 74 iterations, respectively, compared to 23, 47, and 76 iterations for the standard GS rule.

3.3. Random-Feature Coordinate Descent

As a baseline for comparison, we also evaluate coordinate descent with random coordinate selection. At each iteration, a coordinate index is sampled uniformly from $\{1, \dots, d\}$ and updated using the same update rule as in the adaptive methods.

Since this method is stochastic, we repeat the experiment 5 times using different random seeds and report the mean and standard deviation of the loss across runs. Each run is performed for 5000 iterations starting from the same zero initialization.

Although the random selection strategy results in lower per-iteration computational cost due to the absence of gradient scanning, it requires substantially more iterations to achieve comparable loss values. The reported iteration counts for reaching specific loss thresholds correspond to the mean behavior across the five runs.

4. Experimental Results

4.1. Loss Convergence

Figure 1 shows the convergence of the logistic loss as a function of iteration number for the Gauss–Southwell (GS), normalized GS, and random-feature coordinate descent methods. A horizontal dashed line indicates the optimal loss value L^* obtained using the batch logistic regression solver. All three methods reduce the loss to values close to the reference stopping loss L^* within 5000 iterations. For the random-feature method, the curve shown corresponds to the mean loss across 5 independent runs, with error bars indicating ± 1 standard deviation. However, the adaptive coordinate selection strategies exhibit substantially faster convergence than the random baseline in terms of iteration count.

In particular, the normalized GS rule consistently achieves lower loss values in fewer iterations than both the standard GS rule and random selection. For example, the normalized GS method reaches loss values below 1.0, 0.1, and 0.01 after 19, 44, and 74 iterations, respectively. The standard GS rule requires 23, 47, and 76 iterations to reach the same thresholds, while the random method requires 71, 159, and 272 iterations.

Although the random method incurs a lower computational cost per iteration due to the absence of gradient scanning, its slower reduction in loss leads to inferior convergence behavior overall. All methods display similar asymptotic behavior once the loss approaches L^* , consistent with the convexity of the logistic loss function.

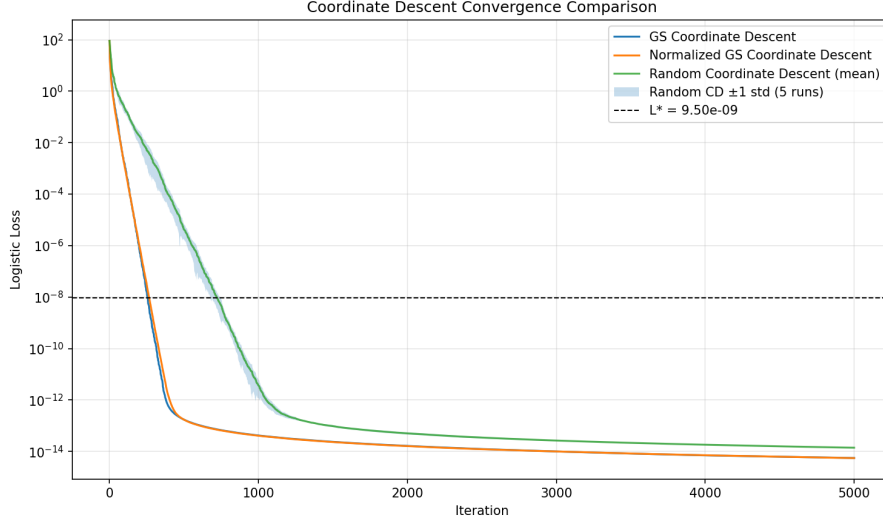


Figure 1. Logistic loss versus iteration for coordinate descent methods. The dashed line indicates the optimal loss value L^* . For random-feature coordinate descent, the solid curve shows the mean loss over 5 runs with different random seeds, and the shaded region indicates ± 1 standard deviation.

4.2. Critical Evaluation

While adaptive coordinate descent methods demonstrate significant improvements over random selection, there remains scope for further enhancement of convergence performance. One possible extension is block coordinate descent, in which multiple coordinates are updated simultaneously at each iteration. This approach can better exploit correlations between features and improve convergence rates in practice.

Another direction is the incorporation of second-order information into the update rule. For example, diagonal or coordinate-wise Newton updates may accelerate convergence by more accurately accounting for local curvature. Alternatively, performing an exact or approximate line search along the selected coordinate direction could improve step size selection beyond the Lipschitz-based update rule used in this work.

Finally, randomized variants of the Gauss–Southwell rule that sample coordinates with probability proportional to the magnitude of their partial derivatives may offer a computationally efficient compromise between fully deterministic and uniformly random coordinate selection.

5. Sparse Coordinate Descent

5.1. k -Sparse Coordinate Method

To encourage sparse solutions, we modify coordinate descent to enforce an explicit cardinality constraint on the parameter vector. Specifically, we maintain an active set $S \subseteq \{1, \dots, d\}$ such that

$$|S| \leq k. \quad (11)$$

Only coordinates in the active set are allowed to have nonzero weights.

At each iteration, a coordinate is selected using the normalized Gauss–Southwell rule over all coordinates. If the selected coordinate i_t already belongs to S , it is updated using the standard coordinate update rule.

If $i_t \notin S$ and $|S| < k$, the coordinate is added to S and updated. If $i_t \notin S$ and $|S| = k$, we perform a swap: the coordinate in S with the smallest absolute weight $|w_j|$ is removed (its weight is set to zero), and i_t is inserted into the active set. The selected coordinate is then updated.

This procedure enforces the sparsity constraint $\|w\|_0 \leq k$ at all iterations while retaining the same coordinate-wise update rule as the unconstrained method.

5.2. Optimality Discussion

The logistic loss $L(w)$ is convex in w . However, the sparsity constraint

$$\|w\|_0 \leq k \quad (12)$$

is non-convex. As a result, the overall optimization problem becomes non-convex, and global optimality is not guaranteed. This is a common limitation in optimization problems involving ℓ_0 constraints, which are inherently non-convex (Blumensath & Davies, 2009).

The proposed algorithm may converge to a local minimum or a suboptimal stationary point. Therefore, while the method enforces the desired sparsity level, it does not necessarily recover the globally optimal k -sparse solution.

5.3. Sparse Experimental Results

We evaluate the sparse coordinate descent method for $k \in \{1, 3, 5, 7, 10, 13\}$, running each configuration for 5000 iterations.

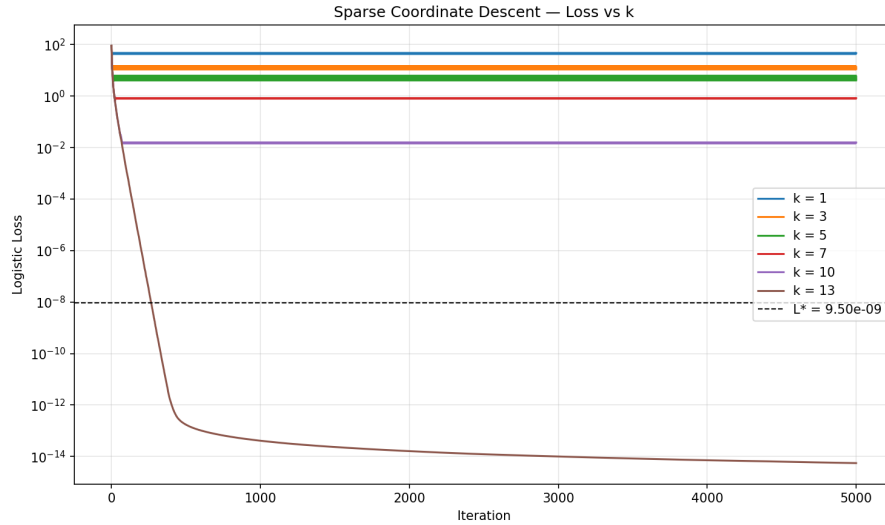


Figure 2. Logistic loss versus iteration for different sparsity levels k . Larger k values allow convergence to lower objective values.

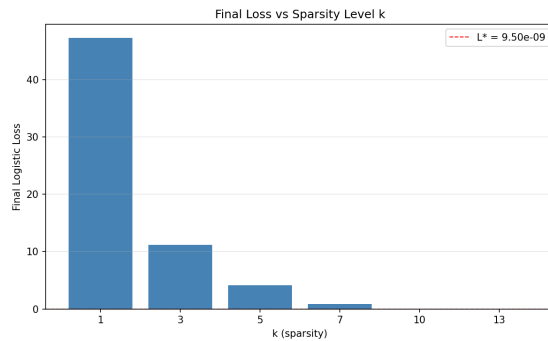


Figure 3. Final logistic loss as a function of sparsity level k . Loss decreases monotonically as more features are allowed.

Convergence behavior. Figure 2 shows the logistic loss as a function of iteration number for each sparsity level. As expected, larger values of k allow the algorithm to achieve lower loss values. When $k = 13$, corresponding to no sparsity constraint, the method recovers the unconstrained optimum L^* . For smaller values of k , the loss decreases initially but plateaus at higher values due to the cardinality constraint.

Final loss comparison. After 5000 iterations, the final logistic loss decreases monotonically as the sparsity level k increases. Figure 3 provides a visual summary of these results. For example, when $k = 10$, the final loss is already very small (approximately 0.0158), and when $k = 13$, the sparsity constraint becomes inactive, yielding a loss that is effectively zero and matching the unconstrained optimum L^* .

This behavior is consistent with the fact that the feasible set

enlarges as k grows. For smaller values of k , the model is forced to exclude potentially informative features, leading to higher objective values.

Notably, the feature corresponding to *Proline* is selected for all values of k , indicating strong discriminative power. As additional coordinates are permitted, the algorithm progressively incorporates complementary features, reducing the training loss accordingly.

6. Conclusion

In this project, we implemented and evaluated coordinate descent methods for minimizing the binary logistic regression loss. In particular, we compared adaptive coordinate selection strategies based on the Gauss–Southwell rule and its normalized variant against a random-feature coordinate descent baseline.

Experimental results show that adaptive coordinate descent achieves significantly faster convergence in terms of iterations than random coordinate selection. While random coordinate descent benefits from lower per-iteration computational cost, it requires substantially more iterations to reach the same objective value. Both the Gauss–Southwell and normalized Gauss–Southwell methods converge rapidly to the optimal loss L^* , with the normalized rule consistently reaching intermediate loss thresholds in fewer iterations due to improved scaling and stability.

We further extended the method to obtain sparse solutions under an explicit cardinality constraint $\|w\|_0 \leq k$. The resulting k -sparse coordinate descent algorithm demonstrates a clear trade-off between sparsity and model performance. As the number of allowed non-zero coefficients increases, the final logistic loss decreases monotonically, with the full model ($k = 13$) recovering the unconstrained optimum. For

smaller values of k , the objective function plateaus at higher values due to the restricted feasible set.

Overall, the results highlight the effectiveness of adaptive coordinate descent for convex optimization problems and demonstrate its flexibility in incorporating sparsity constraints. However, due to the non-convexity introduced by the ℓ_0 constraint, the sparse variant does not guarantee recovery of the globally optimal k -sparse solution.

Software

The implementation is organized into four Python scripts. `data_preprocessing.py` loads the Wine dataset, filters the binary classes, converts labels to $\{-1, +1\}$, standardizes the features, and saves the processed data matrices. `baseline_logistic.py` trains an effectively unregularized logistic regression model using `sklearn` to obtain the optimal reference loss L^* . `coordinate_descent.py` contains the core implementations of logistic loss, gradient computation, and coordinate-wise Lipschitz constants, and runs the Gauss–Southwell, normalized Gauss–Southwell, and random coordinate descent algorithms. Finally, `sparse_coordinate_descent.py` implements the k -sparse extension using an active-set strategy and performs experiments for multiple sparsity levels. All scripts generate intermediate result files and figures used in the experimental evaluation.

AI Usage Statement

AI tools were used to assist in formatting and organizing the code, adding comments, generating the `README.md`, and refining technical writing. No source material or original research content was generated by AI.

References

- Aeberhard, S., Coomans, D., and de Vel, O. Wine recognition data set. UCI Machine Learning Repository, 1994.
- Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Blumensath, T. and Davies, M. Iterative hard thresholding for compressed sensing. *Applied and Computational Harmonic Analysis*, 27(3):265–274, 2009.
- Nesterov, Y. Efficiency of coordinate descent methods on huge-scale optimization problems. *SIAM Journal on Optimization*, 22(2):341–362, 2012.
- Nutini, J., Schmidt, M., Laradji, I., and Friedlander, M. Coordinate descent converges faster with the gauss–southwell rule than random selection. *ICML*, 2015.
- Wright, S. J. Coordinate descent algorithms. *Mathematical Programming*, 151(1):3–34, 2015.